

The Data Access Layer

The Data Access Layer..... 1

Data Access Layer Introduction..... 1

Create Database Structure 2

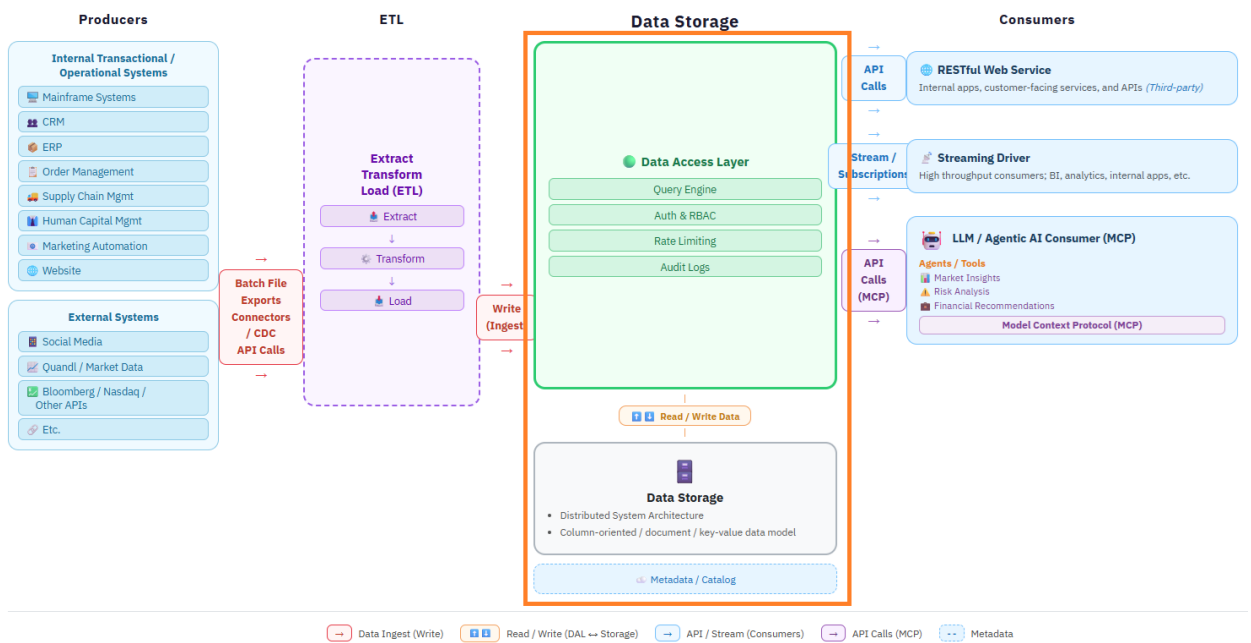
Mapping Database Records to Domain Objects..... 2

Repository Design Pattern..... 2

Implementation Hints..... 3

Expected Deliverables 3

Data Access Layer Introduction



The Data Access Layer (DAL) is a low-level component that offers a clean layer for reading and writing data in your data warehouse. Other modules (for example, the ingestion process and the REST API) should rely on the DAL instead of talking to the database directly.

Why we need a DAL

The DAL hides database-specific details (data model, query language, drivers, serialization) behind a small set of operations. This keeps the rest of your platform easier to evolve, test, and replace.

Learning outcomes

- Identify the responsibilities of the DAL in a data platform.
- Materialize the conceptual model and database model.
- Design a repository-style interface that can be reused across multiple components.
- Implement efficient access to “latest version” data when multiple versions exist for the same logical record.

The next sections should be read in order, as they outline the next steps in your implementation: first materialize the database design in your database management system, map the tables to high-level programming concepts, and leveraging the repository pattern expose the persistence layer to other components of the system.

Create Database Structure

Start from your conceptual and database models define the persistent structures needed by the platform (tables/collections, keys, and constraints). Your physical design must be driven by the queries you need to support.

Design hint: if time series partitions become too large, you can split them by adding a year (or another time bucket) in the partitioning strategy (partition key, usually). This is a simple way to keep partitions bounded while still supporting range queries.

Mapping Database Records to Domain Objects

Applications (Python/Java/.NET etc.) manipulate data using domain objects (e.g., Asset, DataSource, TimeSeriesRecord class), while databases store records/documents/rows. You need a mapping layer that translates between these two worlds.

Technology choices

One may implement mapping in a

- **low-level** way - explicit native queries (Mongo, CQL etc.) executed using DBMS specific drivers, plus manual mapping between domain models and database records, or
- **high-level** way - using *object mapping framework* that can load/store your entities automatically.

The important part is not the framework, but the boundary: the rest of the platform should not care what database or mapping approach you chose.

Repository Design Pattern

The DAL should provide a uniform interface for its clients to access the warehouse. We suggest using the **Repository** design pattern: each entity set has a repository that exposes a small, consistent set of operations.

Generic repository interface

```
warehouseRepository<E, K>
  save(entity: E) -> E
  delete(entity: E) -> void
  deleteAll(partitionKey: K) -> void
  findLatest(partitionKey: K) -> E
  findAll(partitionKey: K) -> Iterable<E>
```

where E stands for Entity (e.g., Asset, Data Source etc.) and K stands for partition key datatype of the entity E. The method findLatest(partitionKey) returns the newest version (i.e. most recent system time) when multiple versions exist for the same logical key. Your database design should make this operation efficient (avoid scanning multiple partitions).

Implementation Hints

- Keep the DAL as the only place where database queries exist. This is your “single source of truth” for persistence logic and operations.
- Model versioned records explicitly. Support fetching the latest record from a partition.
- Make read paths deterministic: define ordering rules (e.g., newest first) and apply them consistently.
- Prefer idempotent writes where possible (e.g., re-running ingestion/ETL processes should not corrupt data).

Expected Deliverables

- A DAL module (library/package) with repositories for your core entities (at least: Asset, DataSource, TimeSeriesData).
- A small set of unit tests that prove: save → findLatest → findAll works as intended.